

JavaScript Data Types

Data is the most important part of any programming language. Understanding how data is stored and in which format allows us to modify it more effectively. In JavaScript, data types are broadly classified into two categories:

- **Primitive Data Types**
- **Non-Primitive Data Types**

Primitive Data Types

Primitive data types are the most basic types of data. They are immutable, meaning their values cannot be changed once they are set. JavaScript has the following primitive data types:

1. String

A string is a sequence of characters. It can contain letters, numbers, or symbols enclosed in either single, double, or backticks.

Example:

```
let name = "Yogesh";
console.log(typeof name); // Output: string
console.log(name);        // Output: Yogesh
```

2. Number

A number represents both integer and floating-point values.

Example:

```
let height = 180;
console.log(typeof height); // Output: number
console.log(height);        // Output: 180
```

3. Boolean

A boolean represents one of two possible values: `true` or `false`.

Example:

```
let isActive = true;
console.log(typeof isActive); // Output: boolean
console.log(isActive);        // Output: true
```

4. Null

The `null` type is used to represent the absence of any value or object.

Example:

```
let address = null;
console.log(typeof address); // Output: object (This is a known
console.log(address);       // Output: null
```

5. Undefined

A variable that has been declared but not assigned a value is of type `undefined`.

Example:

```
let lastName;
console.log(typeof lastName); // Output: undefined
console.log(lastName);       // Output: undefined
```

6. BigInt

The `BigInt` type is used to represent large integers that cannot be represented by the `Number` type.

Example:

```
let bigNumber = 1234567890123456789012345678901234567890n;
console.log(typeof bigNumber); // Output: bigint
console.log(bigNumber);       // Output:
1234567890123456789012345678901234567890n
```

7. Symbol

A symbol is a unique and immutable primitive value used primarily as an identifier for object properties.

Example:

```
let sym = Symbol('description');
console.log(typeof sym); // Output: symbol
console.log(sym);       // Output: Symbol(description)
```

Note:

- All primitive data types, except `null` and `undefined`, have *wrapper objects*. A wrapper object is an object that wraps a primitive value so that we can use methods and properties on them.

What is a Wrapper Object?

A wrapper object allows primitive values like strings, numbers, and booleans to behave like objects. For example, a string is a primitive, but JavaScript will automatically wrap it in a `String` object when you call methods on it.

Example:

```
let str = "Hello";  
console.log(str.toUpperCase()); // Output: HELLO
```

Dynamic Typing in JavaScript

JavaScript is a **dynamically typed language**, meaning that you don't need to specify the type of a variable when declaring it. The type of a variable is determined at runtime based on the value it holds.

Checking the Type of a Variable

If you want to check the type of a variable, you can use the `typeof` operator. Here's how it works:

```
let fname = "Yogesh";  
console.log(typeof fname); // Output: string
```

JavaScript Variable Declaration Keywords

In JavaScript, there are three main ways to declare variables:

1. **let**
2. **const**
3. **var**

Let's look at each one in more detail.

1. **let** — Block-scoped variable

The `let` keyword is used to declare variables that are **block-scoped**. This means that the variable is only accessible within the block (enclosed by `{ }`) where it was declared.

- **Reassignable:** You can reassign values to a variable declared with `let`.
- **Block-scoped:** Its value is accessible only within the block where it's defined (e.g., inside a loop or function).

Example:

```
let age = 25;
age = 30; // Reassigning the value is allowed
console.log(age); // Output: 30
```

2. `const` – Block-scoped constant

The `const` keyword is used for variables that **cannot be reassigned** once their value is set.

- **Reassignable:** `const` variables cannot be reassigned. If you try to change their value, it will throw an error.
- **Block-scoped:** Like `let`, `const` is also block-scoped.

Example:

```
const pi = 3.14;
// pi = 3.1415; // Error: Assignment to constant variable.
console.log(pi); // Output: 3.14
```

Note: Even though the value can't be reassigned, if the `const` is used with an object or array, the content of the object/array can be modified.

```
const person = { name: "John", age: 30 };
person.age = 31; // This is allowed because we are modifying the contents
of the object.
console.log(person); // Output: { name: 'John', age: 31 }
```

3. `var` – Function-scoped variable

The `var` keyword is the **oldest** way of declaring variables in JavaScript. However, it is **function-scoped**, meaning it is only accessible within the function it was declared in (if declared inside a function). Unlike `let` and `const`, `var` has **function scope** rather than block scope.

- **Reassignable:** You can reassign values to a variable declared with `var`.
- **Function-scoped:** Variables declared with `var` are available throughout the entire function.

Example:

```
var city = "New York";
city = "Los Angeles"; // Reassigning the value is allowed
console.log(city); // Output: Los Angeles
```

Key Differences Between `let`, `const`, and `var`:

Feature	<code>let</code>	<code>const</code>	<code>var</code>
Scope	Block-scoped	Block-scoped	Function-scoped
Reassignable	Yes	No (constant)	Yes
Hoisting	Yes, but not initialized	Yes, but not initialized	Yes, but initialized to <code>undefined</code>

When to Use Which?

- Use `let` when you expect the variable value to change.
 - Use `const` when the variable value should remain constant.
 - Use `var` only if you need to support older JavaScript code or for backward compatibility. In modern JavaScript, `let` and `const` are generally preferred over `var`.
-

Non-Primitive Data Types

Non-primitive data types, also known as **reference data types**, are more complex than primitive data types. These include:

- **Objects**
- **Arrays**
- **Functions**

In JavaScript, non-primitive data types are treated as objects. Interestingly, JavaScript allows the creation of objects directly without using a class.

Example of an Object:

```
let person = {
  fname: "Yogesh",
  lname: "Pandian",
  height: 180
};

console.log(typeof person); // Output: object
console.log(person);        // Output: { fname: 'Yogesh', lname:
'Pandian', height: 180 }
```

Key Takeaways:

- **Primitive Data Types:** Simple values like strings, numbers, booleans, etc. They are immutable.
 - **Non-Primitive Data Types:** More complex structures like objects, arrays, and functions. They are reference types and can be modified.
 - JavaScript is **dynamically typed**, so you don't need to declare a variable's type explicitly.
 - Use `typeof` to check the type of a variable.
 - Variables can be declared with `let` (mutable) or `const` (immutable).
-